W6 Code Guide: Open the Definition Before Calling It

All links use the fixed `2026-fall-w6` revision and point to actual definition lines. The notebook downloads and verifies SHA-256 hashes automatically. W2 graph code is reused; W6 discrete samplers are new and do not silently substitute the W5 Gaussian samplers.

Array order matters: a path is `[z1,z2,z3]`; path tables are lexicographic; marginal columns are state zero then state one. Parameters are fixed, so there is no fit method. The exact routines are intentionally limited to small models.

PairwiseModel

Finite undirected model represented by log potentials, not fitted parameters. log_unary: numeric (N,K) array, N nodes and K states per node, N,K >= 1. log_edges: dict {(u,v): (K,K) numeric array}, indexed [state_u,state_v]. Node IDs are 0..N-1 and states are 0..K-1. Reverse keys are transposed to canonical order; duplicate undirected edges and self edges are invalid. The unnormalized log joint sums one unary entry per node and one edge entry per edge. Factors need not be normalized probability tables. Negative infinity encodes a hard zero; NaN and positive infinity raise ValueError. Arrays are copied at construction. Methods perform inference/conditioning; no training data or fit method is involved. Example: PairwiseModel(np.zeros((2,2)), {(0,1):np.log([[2.,1.],[1.,2.]])}).

enumerate_exact

Enumerate the complete joint as an independent small-model oracle. model: PairwiseModel. max_states: maximum K**N assignments, default 262144. Returns dict: marginals (N,K) normalized probabilities; log_z float; states (K**N,N) integer assignments; probabilities (K**N,) normalized joint weights in corresponding order. No input is mutated. Excess states or zero total mass raise ValueError. Runtime O(K**N*(N+E)); joint storage is exponential. Use for verification, not large graphs. For clamped models, log_z retains unnormalized evidence mass and is not generally log P(e).

tree_sum_product

Run exact two-pass log-domain sum-product on a connected tree. model: PairwiseModel (single node allowed). root: starting node, default 0. Returns dict marginals (N,K), log_z float, and log_messages dict whose (u,v) entry has K values indexed by the receiver v's state. Messages retain scale so log_z is recoverable. Model is not changed. Invalid root, a cycle/disconnected graph, or zero total mass raises ValueError. Time O(N*K**2) at bounded degree, with O(K*sum(degree**2)) extra work from explicitly recomputing neighboring products. Different roots should produce the same marginals up to floating-point tolerance.

SensorModel

Fixed binary HMM: observations tuple, stay/accuracy/prior_one in (0,1).

Defaults are y=(1,0,1), stay=.8, accuracy=.8, prior_one=.5. No fitting occurs. A state is a length-T binary path; T must be between 1 and 10. Invalid/nonfinite probabilities or nonbinary observations raise ValueError.

SensorModel.pairwise

Return W2 PairwiseModel with unary (T,2), transition edges (2,2).

First unary includes the prior; each unary includes its emission. Therefore the sum of joint weights is P(y), not an arbitrary scale.

SensorModel.log_joint

Return log P(z,y) for binary paths of shape (N,T); output (N,).

Uses the directed HMM product, independently of W2 enumeration. Rejects invalid shape/values; no normalization and no mutation.

exact_posterior

Return W2 oracle states, probabilities, marginals and log_z plus log_joint.

States (2**T,T) are lexicographic. probabilities (2**T,) sum to one. marginals (T,2) use columns zero/one. Exponential reference only.

exact_filter

Independent forward recursion, O(T): P(z_t=1 | y_1:t), shape (T,).

Also returns predicted probabilities (T,2), normalizers (T,), and log_z. A prefix never uses a future observation. This is filtering, not smoothing.

product_probability

Evaluate independent Bernoulli q on paths (N,T), parameters (T,).

Parameters must lie strictly inside (0,1). Returns (N,), no normalization over a subset. On all binary paths these values sum to one.

mean_field

Sequential coordinate-ascent VI for q(z)=product Bernoulli(phi_t).

initial in (0,1) initializes every factor. sweeps is a positive integer. Returns phi (T,), probabilities (2**T,), elbo (initial + each sweep), reverse_kl, converged, sweeps. Each update averages log joint over OTHER factors then normalizes. This finite oracle implementation is educational. Convergence of coordinates does not certify a global family optimum.

distribution_summary

Summarize a normalized path distribution: last-state event and covariance.

states (N,T), probabilities (N,). Outputs probability_last_one, covariance_first_last, posterior_sd_last (spread of Bernoulli state). This SD is NOT the Monte Carlo standard error. T=1 yields variance.

mh_transition

Exact lazy one-bit MH transition matrix (2**T,2**T), row stochastic.

Choose uniformly among T bit flips and one self proposal. Symmetric proposal, acceptance min(1,p_new/p_old); all rejection mass stays on diagonal. Positive model probabilities ensure connected paths and self transitions.

sample_paths

MC oracle, self-normalized IS, lazy MH, or full-sweep Gibbs on fixed y.

n positive draws; burnin nonnegative sweeps/steps for chains only. IS uses independent Bernoulli proposal_one in (0,1). Returns paths (n,T), weights (n,), histogram probabilities (2**T,), weight_ess for IS only, and move_fraction for chains. Rejected MH proposals remain as repeated rows. MC needs the exact table: it is a benchmark, not a scalable inference method. weight_ess is a weight-concentration measure, not MCMC ESS or unique states.

particle_filter

Bootstrap filter with resampling before each next transition, O(N*T).

Return prob_one (T,), weight_ess (T,), particles/weights (T,N), ancestors (T,N; -1 at t=0). Each reported estimate is weighted BEFORE resampling. The target at t is P(z_t | y_1:t). Distinct ancestor IDs differ from state values: many distinct particles can all be in state one. No future data.

static_experiment

Return Figure, JSON-safe metrics dict, and CSV path for a fixed target.

method: Exact/VI/MC/IS/MH/Gibbs. n: samples or max VI sweeps. Model has y=(1,0,1), accuracy=.8. extra is an optional callable(metrics)->dict; use it to add a student diagnostic without editing the model or sampler. Temporary CSV contains target settings, seed, budget and exact reference.

sequential_experiment

Return Figure, per-time DataFrame, CSV for SMC vs exact FILTERING.

n positive particle count, seed integer, stay in (0,1). No smoother reference is used. CSV has one row per observed time, not per MCMC step.

create_app

Build (do not launch) a gradio.Blocks workbench; optional extra callback.

Use demo.launch(share=True) in Colab only while you need a temporary URL. create_app itself starts no server. Both experiments are independently runnable.